

The AI Product Manager Playbook

From "I can write skill specs" to "I can architect, evaluate, and defend agentic AI systems" - every concept taught twice: once in plain English, once as something you already shipped in Prodigy.

Prepared for Kaushik - Product Manager, Tachyon CLM / Prodigy - September 2026

8 phases

Foundations -> AI PM career

45+ concepts

Plain English + Prodigy mapping

2 source docs

Ground-Up Roadmap + PC AI Architecture

1 real portfolio

Built from your own PRDs & interviews

ORIENTATION

How to use this playbook

You already know how to scope skills, write PRDs, and run user interviews for an agentic product. This rework upgrades that craft into the vocabulary that AI Product Manager interview loops actually test - foundations, architecture, orchestration, evals - and then turns your real Prodigy work into the portfolio that gets you hired.

1. **Read one phase per sitting.** Each phase ends with a short quiz - cover the Answer box with your hand, then check yourself.

2. **Every concept card has the same four parts.** Plain English -> Analogy -> Concrete example -> what it means in Prodigy (+ an interview one-liner or AI-PM angle).
3. **Use Regalia Gold US / Alex as your default story world.** Interviewers remember concrete products; they forget abstract "AI platforms."
4. **Phase 5 is your proof.** If you can narrate Path 1 -> CR -> dry run -> human deploy in agent vocabulary without overclaiming, you already think like an AI PM.
5. **Phase 6 is the career plan.** It turns this exact repo - your interviews, your synthesis doc, your architecture decision - into portfolio artifacts.

KEY TAKEAWAY

Prodigy is not "a chatbot on Product Center." It is an **embedded agent harness**: skills are procedures, tools are typed adapters to the same services as REST, MCP is for external harnesses, chat *proposes*, forms *own truth*, and the change-request workflow is the only path to apply. Schema is law. Humans approve. That single sentence is also, almost word for word, what a hiring panel wants to hear from an AI PM candidate.

What this is not

- Not a substitute for SPI intakes or the actual Product Center AI architecture decision - it explains that decision, it doesn't replace it.
- Not permission to claim models "learn" from chatting with Alex.
- Not a license to ship unsupervised deploy. Propose $\neq$ apply - in Prodigy or in your interview answers.

What this playbook synthesizes

Three inputs, one spine. Every concept below is tagged with where it came from, so you always know if you're citing a general AI concept, a Zeta architecture decision, or a Prodigy-specific mechanism.

Ground-Up Roadmap

The Ground-Up AI Roadmap

A 24-week, 6-phase personal curriculum for two target seats - AI Product Manager and AI Forward Deployed Engineer. Supplies the concept spine (foundations -> building blocks -> applied architectures ->

adjacent & advanced -> role tracks -> proof of work) and the "what a hiring loop actually tests" framing used in Phases 4 and 6 here.

Architecture decision

Product Center AI Architecture

The real, approved decision doc for Prodigy: MCP API, Product Center Skills, the AI REST API, ToolRegistry, Agents SDK on Azure Foundry, tracing, and evaluation. Used here as the running case study for "applied architecture" - the part of the roadmap that is usually taught with a toy example, taught here with your actual system instead.

Prodigy-native

Your own Prodigy specs, interviews, and synthesis

SPI-24205 through 24671, the HDFC / Sparrow / Ryze interview records, and the cross-issuer synthesis you already shipped this week. These are not homework - Phase 6 turns them directly into your AI PM portfolio.

The map - 8 phases

Each phase builds one layer of fluency. Skip Phase 0 only if you can already explain tokens, embeddings, and training vs. inference without hand-waving - most PMs who "know AI" still fail that bar.

PHASE 0

Foundations

AI->ML->DL->GenAI->LLM nesting, how models learn, what's inside an LLM: tokens, embeddings, attention, context.

PHASE 1

Chatbot -> Agent

Decision framework, the agent loop, limits, when NOT to agentize, human-in-the-loop.

PHASE 2

Skills - Tools - MCP

RAG, vector DBs, fine-tuning, and the real Product Center AI architecture as one worked case study.

PHASE 3

Orchestration

Routing, handoffs, multi-agent patterns, loop vs. graph, cost / latency / blast radius.

PHASE 4

Trust, evals, infra

Grounding, guardrails, eval design (the highest-signal AI PM skill), unit economics.

PHASE 5

Prodigy, worked

Paths 1-3 as a real agent system: proof you can narrate architecture end to end.

PHASE 6

Becoming an AI PM

What the role is graded on, the interview loop shape, and a portfolio plan built from work you've already shipped.

PHASE 7

Interview ammo

60-90 second spoken answers, trap questions, a bank-safe narrative you can say out loud.

PROGRESSION RULE

If you can't pass a phase's quiz without looking, don't advance - interviewers will probe the exact layer you skipped.

PHASE 0 - FOUNDATIONS

What AI - and an LLM - actually is

Goal: stop sounding like "AI magic" and start sounding like someone who ships AI inside a system of record.

Before agents, tools, and MCP, you need a clean mental model of the model itself. Everything Prodigy does that feels clever is still next-token prediction, constrained by context, tools, schema, and human confirmation.

0A - What kind of intelligence is this

Foundational

Ground-Up Roadmap

The nesting: AI -> ML -> DL -> GenAI -> LLM

WHAT THIS LOOKS LIKE

ARTIFICIAL INTELLIGENCE

any system doing "intelligent" tasks - rules included

↓ contains

MACHINE LEARNING

learns patterns from data

↓ contains

DEEP LEARNING

many-layered neural nets

↓ contains

GENERATIVE AI

produces new content

↓ contains

LARGE LANGUAGE MODELS

text-native generator, e.g. what Prodigy calls

Each box only lives inside the one above it. Prodigy's validation engine is **AI, not ML** - a hand-written rule. Prodigy's chat is **all five boxes deep**.

PLAIN ENGLISH

Artificial intelligence is the broad goal of machines doing useful "intelligent" work - including hand-written rules. Machine learning is AI that learns patterns from data instead of being explicitly programmed. Deep learning is ML using many-layered neural networks. Generative AI is deep learning that produces new content instead of a label or a number. Large language models are generative models specialised in text - predicting the next token, steerable with prompts, tools, and retrieval.

ANALOGY

Nested boxes: transport -> motor vehicles -> cars -> electric cars -> Teslas. Every Tesla is an electric car; not every motor vehicle is. When a vendor says "we use AI," they've told you as much as "we use transport" - the useful question is which box down the stack they're actually in.

CONCRETE EXAMPLE

Four systems in one bank: a rules engine that flags any transaction over \$10,000 (AI, no learning) - a gradient-boosted model predicting which cards will default (classical ML, not deep learning) - a neural net reading scanned KYC documents (deep learning, discriminative) - Prodigy drafting a change-request summary (LLM, all the way down the stack). All four are legitimately "AI." Only one needs a GPU bill and a context window.

In Prodigy

Prodigy sits on Product Center, a system of record. The LLM is the language and planning layer; blueprints, fields, entitlements, and the CR workflow remain non-negotiable product contracts - not model opinions.

Interview one-liner

"LLMs are a generative subset of AI; in Prodigy they compose and plan, but schema and services remain the source of truth."

New in this rework

Ground-Up Roadmap

Supervised, unsupervised, and reinforcement learning

PLAIN ENGLISH

Supervised learns from labelled examples ("here are 50,000 emails marked spam / not spam"). **Unsupervised** finds structure with no labels ("here are 50,000 customers - what natural groups exist?"). **Reinforcement** learns from trial, reward, and penalty. Modern LLMs use all three in sequence: unsupervised pre-training on huge text, supervised fine-tuning on curated examples, then reinforcement learning from human feedback (RLHF) to align tone and safety.

ANALOGY

Supervised is a tutor marking your homework. Unsupervised is sorting a box of mixed photos into piles with no labels provided. Reinforcement is learning to ride a bike - no one tells you the exact muscle sequence, only whether you fell over.

CONCRETE EXAMPLE

Product Center's **validation framework** (SPI-24209) is a supervised, rules-based classifier in spirit: fixed labels ("blocks release" / "warns" / "passes") applied deterministically. When Prodigy explains *why* a fee is out of range in plain language, that explanation layer is generative - but the pass/fail decision underneath is not learned at all; it's a rule.

In Prodigy

Validation is deterministic (supervised-style rules, hand-authored). The NLP summary layer on top is generative. Never let the generative layer decide the pass/fail - that's the classic AI-PM failure mode.

Interview one-liner

"I know the difference between a system that learned a decision boundary and one that generates an explanation of a hand-written rule - and I never let the second one pretend to be the first."

New in this rework

Parameters, weights, and "the model"

PLAIN ENGLISH

A model is, physically, a very large file of numbers called parameters (or weights). Architecture defines how those numbers are arranged and combined; training finds good values for them. A "70-billion-parameter model" is literally 70 billion numbers. There is no database of facts inside - no lookup table of your fee schedule.

ANALOGY

A weights file is like a chef's trained palate, not a recipe book. The chef can improvise a plausible dish from anything you describe - but if you want an exact recipe from your own kitchen, you hand them the actual card (retrieval / tools), you don't trust their memory of "a dish kind of like this."

CONCRETE EXAMPLE

Ask a raw model "What is Regalia Gold US's joining fee?" and it will confidently guess a plausible number - because that number was never in its weights. Ask the same question through Prodigy's Path 3 (cited Q&A), and the tool call fetches the real value from Product Center / GitDB; the model only phrases the sentence around it.

In Prodigy

Every product-specific fact must come from a tool call or a citation, never from weights. This is the entire reason ToolRegistry and grounding exist.

Interview one-liner

"Weights are a trained palate, not a filing cabinet - anything customer- or product-specific has to be retrieved, not recalled."

Prodigy-native

Training vs. inference (why chatting doesn't teach Prodigy)

WHAT EACH PROCESS LOOKS LIKE

TRAINING (HAPPENS ONCE, OFFLINE)

Huge dataset

Expensive compute, days/weeks

Weights permanently updated

INFERENCE (HAPPENS EVERY CHAT)

Alex's message this turn

Frozen weights + tools + context

One answer, weights unchanged

Alex correcting a fee in chat is the **right-hand column, every time** - nothing on the left ever runs during a normal conversation.

PLAIN ENGLISH

Training updates model weights using large datasets and expensive compute. Inference is using a frozen (or lightly adapted) model to generate outputs for a request. When Alex chats with Prodigy, he is doing inference. The model does not permanently learn Regalia Gold's fee from that conversation unless you build an explicit memory or data pipeline - and even then you're writing to your own systems, not "retraining GPT."

ANALOGY

Training is rewriting the textbook. Inference is a student answering today's exam using the textbook plus the open-book materials you hand them (tools, schema, retrieval).

CONCRETE EXAMPLE

Alex corrects "joining fee is \$95, not \$90." Confirm patches the form / draft. Future answers should come from Product Center state, not from the model's vibes about yesterday's chat. Conversation history helps continuity; it is not a weight update.

In Prodigy

Never tell an interviewer "the agent learns from our users" without specifying session memory, eval datasets, fine-tunes, or preference data - and the governance around each.

Interview one-liner

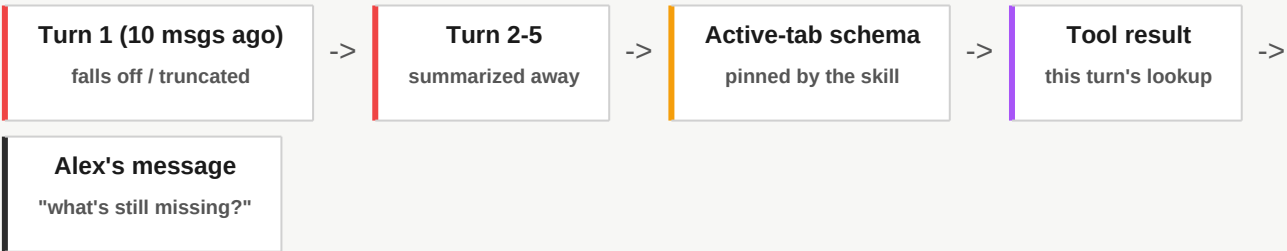
"User chat is inference against tools and schema; learning happens through product data and deliberate eval/train loops, not by osmosis."

0B - What's inside an LLM

Prodigy-native

Tokens and context windows

WHAT A CONTEXT WINDOW LOOKS LIKE WHEN IT FILLS UP



The whiteboard is full. Page-aware context engineering means the **schema and tool result** stay pinned every turn - old chat turns are what gets dropped first, not the fields that matter.

PLAIN ENGLISH

Models don't read "words" the way you do - text is broken into tokens (subword pieces). The context window is the maximum tokens the model can attend to in one call: instructions, history, tool results, and schema snippets all compete for that budget. Exceed it and something gets truncated or summarized away.

ANALOGY

A whiteboard with a fixed size. Skills, page context ("Fees tab open"), conversation history, and tool outputs all fight for marker space. When the board fills up, earlier notes get erased.

CONCRETE EXAMPLE

Guided configuration on a fat blueprint: dump the entire schema, full MITC text, and twenty turns of chat into one call and you blow the window - the model misses a required field. Product design must choose what's pinned (schema for the active tab), what's retrieved on demand (a lookup list), and what's summarized.

In Prodigy

Page-aware Prodigy is a context-engineering product: pass page, product, blueprint, active tab, and session id into the skill - don't force Alex to restate what's on screen.

Interview one-liner

"Context is a scarce product resource; we design what the agent sees, not just what the user types."

New in this rework

Ground-Up Roadmap

Embeddings - meaning as coordinates

WHAT AN EMBEDDING SPACE LOOKS LIKE

"joining fee"

"annual fee waiver"

"no yearly charge"

"credit limit"

"spend cap"

fee-related meaning cluster credit-limit meaning cluster

"Annual fee waiver" and "no yearly charge" share **no words** but land close together - because embeddings encode meaning, not spelling. Keyword search would miss this; embeddings wouldn't.

PLAIN ENGLISH

An embedding turns a piece of text into a list of numbers (a vector) positioned in a high-dimensional space, such that texts with similar *meaning* land near each other - even if they share no words. This is the foundation for search, recommendation, clustering, and retrieval-augmented generation (RAG).

ANALOGY

Think of a giant map where every sentence gets a pin. "Annual fee waiver" and "no yearly charge" get pinned close together even though they don't share a word - because they mean roughly the same thing. Keyword search would put them on opposite sides of the map; embeddings put them next to each other.

CONCRETE EXAMPLE

If Prodigy ever needs to find "which existing product is most like the one Alex is describing" for clone-and-merge (SPI-24207), embeddings are how you'd compare a free-text description against product summaries - not by matching keywords, but by matching meaning.

In Prodigy

Today, clone-and-merge uses explicit product selection (Alex picks the product), which is simpler and more auditable than semantic search. Embeddings become relevant if you ever build "find me a similar product" as a suggestion feature.

Interview one-liner

"Embeddings let us search by meaning instead of by keyword - the substrate under RAG, semantic clone-suggestion, and dedup."

New in this rework Ground-Up Roadmap

Transformers and attention

WHAT ATTENTION LOOKS LIKE WHEN ANSWERING "WHAT'S THE JOINING FEE?"



Bar height = how much weight the model gives each token while generating its answer. "Joining" and "fee" dominate; "is" and "?" barely register. This re-weighting, done per token, is what "attention" means.

PLAIN ENGLISH

The transformer is the architecture that made today's LLMs possible. Its key trick is **attention**: for every token being generated, the model computes how much "weight" to give every other token in context, so it can decide what actually matters right now instead of reading everything with equal importance.

ANALOGY

Attention is like a courtroom where every witness statement gets weighed differently for each specific question - the same testimony matters a lot for one question and almost nothing for another. The model re-weighs the whole room for every word it writes.

CONCRETE EXAMPLE

When Alex asks "what's still missing on the Fees tab?", attention is why the model can pull from the schema tool result (this tab's fields) far more heavily than from an unrelated earlier turn about the Rewards tab - as long as both are still inside the context window.

In Prodigy

You don't need to implement attention - you need to know it exists so you can reason about why irrelevant context (a stale 20-turn-old message) can still distract the model, and why keeping context tight and well-labeled genuinely improves accuracy.

Interview one-liner

"I don't train transformers, but I know attention is why context hygiene is a product lever, not just an engineering nicety."

Prodigy-native

Non-determinism and sampling

PLAIN ENGLISH

Given the same prompt, an LLM can produce different outputs because decoding samples from a probability distribution (temperature, top-p, etc). Even at temperature 0, ties and infrastructure variance can surprise you. Treat outputs as proposals, not database writes.

ANALOGY

Ask three analysts the same open question and you get three wordings. If your bank needs one fee value, you don't let three analysts write the ledger - you pick a process that confirms a single value.

CONCRETE EXAMPLE

Alex says "set joining fee to something reasonable." Two runs may suggest \$95 and \$100. That's fine as a suggestion. It's unacceptable as an unsupervised write. Confirm-before-write and schema enums collapse the ambiguity.

In Prodigy

Non-determinism is why Prodigy separates propose vs. confirm, why evals use golden conversations with tolerances, and why the validation framework is deterministic while NLP summaries can be generative.

Interview one-liner

"We assume the model will vary; product safety comes from confirmation, schema, and deterministic validators - not from hoping temperature is zero."

Prodigy-native

Prompting vs. context engineering

PLAIN ENGLISH

Prompting is writing instructions for one call. Context engineering is the system design of everything the model sees over time: skills, retrieved facts, tool results, entitlements, page state, conversation memory, and what's deliberately withheld. Mature products win on context engineering, not witty system prompts.

ANALOGY

Prompting is writing a sticky note. Context engineering is designing the entire briefing pack: which docs, which APIs, which permissions, which prior decisions, which red lines.

CONCRETE EXAMPLE

A clever prompt saying "always use schema" fails if the schema isn't in context and there's no tool to load it. The durable fix is a ToolRegistry call that loads tabs, fields, enums, and conditionals for the blueprint - then a skill that says confirm before patch.

In Prodigy

Skills are context-engineering artefacts: name, description, procedure, confirmation points, pinned version. They guide procedure; they don't invent product rules or grant access.

Interview one-liner

"I treat skills and tool results as the real product surface; prompts are the thin instruction layer on top."

New in this rework

Ground-Up Roadmap

When classical ML still wins

PLAIN ENGLISH

LLMs are expensive, slow, and non-deterministic relative to a simple rule or a small classical model. If the input is structured, the answer needs to be exact, and the decision boundary is learnable from tabular data, a rules engine or gradient-boosted model usually beats an LLM on cost, latency, and auditability.

ANALOGY

You don't call a consultant to check if a number is bigger than a threshold. A calculator is faster, cheaper, and never hallucinates.

CONCRETE EXAMPLE

"Is this late fee within the legally allowed range?" is a deterministic comparison against a rule table - exactly what Prodigy's validation framework does. Routing that question to an LLM would be slower, costlier, and occasionally wrong. The LLM's job is to *explain* the verdict in plain language, not compute it.

In Prodigy

The entire "schema is law, validation is deterministic, LLM explains" design is a direct application of this principle at product scale.

Interview one-liner

"Being a good AI PM means knowing when to say no to an LLM - the validation engine in Prodigy is deliberately not one."

Exit quiz - Phase 0

1. Which nesting is correct?

1. LLM -> GenAI -> DL -> ML -> AI

2. AI -> ML -> DL -> GenAI -> LLM

3. ML -> AI -> LLM -> GenAI

4. GenAI -> AI -> ML -> LLM

B. Each is a strict subset of the one containing it.

2. Alex chats for an hour correcting fees. What did the base model's weights do?

1. Updated permanently with Regalia Gold knowledge

2. Nothing permanent - this was inference

3. Fine-tuned automatically every 10 turns

4. Stored the weights in GitDB

B. Unless you built a separate memory/train pipeline, chat is pure inference.

3. Why do embeddings matter for a feature like "suggest a similar product to clone"?

1. They store the exact fee values

2. They let you compare meaning, not just keywords

3. They replace the validation framework

4. They make the model deterministic

B. Embeddings enable semantic (meaning-based) similarity search.

4. A late-fee-limit check should be built as:

1. An LLM prompt asked every time

2. A deterministic rule; the LLM only explains the result

3. A fine-tuned model

4. A vector search

B. Classical logic wins on exactness, cost, and auditability; LLMs are for language, not law.

PHASE 1 - FROM CHATBOT TO AGENT

Chatbot, workflow, or agent - and how to decide

Goal: a repeatable decision framework you can defend under pressure, not a vibe.

AI PM must-know

Chatbot vs. workflow vs. agent

WHAT EACH ONE LOOKS LIKE END TO END

CHATBOT

"What's the fee?"

one lookup

answer

WORKFLOW

fee submitted

fixed rule checks

pass / block, same every time

AGENT

"launch a card from this PDF"

unknown # of tool calls

draft, then confirm

The number and order of steps is **fixed** for a workflow, **one-shot** for a chatbot, and **decided by the model, step by step** for an agent.

PLAIN ENGLISH

A **chatbot** converses but doesn't reliably operate tools across multiple steps. A **workflow** is a fixed, deterministic sequence - same input, same path, every time. An **agent** is a goal-seeking loop: it decides which tool to call next based on what it observes, under uncertainty, until a goal or stop condition is met.

ANALOGY

A chatbot is a knowledgeable receptionist. A workflow is a factory conveyor belt. An agent is a project manager who improvises the next step based on what just happened, but still reports to a boss before signing anything.

CONCRETE EXAMPLE

"What's Regalia Gold's joining fee?" -> chatbot-shaped (one lookup, answer). "Every fee change needs the same five validation checks" -> workflow-shaped (fixed, deterministic). "Launch a credit card in the US from this MITC PDF" -> agent-shaped (unknown number of steps, tool choices, and clarifying questions).

In Prodigy

Path 3 (cited Q&A) is chatbot-shaped. The validation framework is workflow-shaped. Path 1 (launch) is agent-shaped - that's exactly why it needs a full skill, not a prompt.

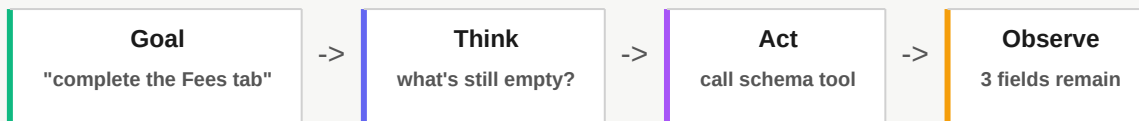
Interview one-liner

"I use a simple test: is the path fixed, is the write dangerous, is language the interface? That tells me chatbot, workflow, or agent."

Foundational

The agent loop

WHAT THE LOOP LOOKS LIKE INSIDE GUIDED CONFIGURATION



■ repeat until tab is complete, Alex exits, or a stop condition fires

Every "agentic" Prodigy feature is this same four-step loop with a different goal, a different tool set, and different stop conditions.

PLAIN ENGLISH

Goal -> think -> act (call a tool) -> observe (read the result) -> repeat, until the goal is reached or a stop condition fires. Every "agentic" feature is this loop with different tools and different stop conditions.

ANALOGY

A detective: form a hypothesis, go check one clue, update the theory based on what you found, decide whether to check another clue or close the case.

CONCRETE EXAMPLE

Guided configuration: goal = "complete the Fees tab" -> think "what's still empty?" -> act: call schema tool -> observe: three required fields remain -> ask Alex or propose from context -> repeat until the tab is complete or Alex exits.

In Prodigy

The loop runs inside a skill with a bounded tool set (ToolRegistry entries for that skill only) and hard stop conditions: max turns, confirmation required before any write, and no autonomous CR submission.

Interview one-liner

"An agent in our product is a bounded loop - not an open-ended chatbot with production keys."

AI PM must-know

Limits and stop conditions

PLAIN ENGLISH

An agent left unbounded can loop forever, spend unbounded tokens, or take unbounded time. Product design must set: max turns, max tool calls, wall-clock timeout, and "fail closed" behaviour (deny rather than guess) when something goes wrong.

ANALOGY

A test drive has a mileage cap and a return time - not because the car is untrustworthy, but because unbounded trust is bad design regardless of trust level.

CONCRETE EXAMPLE

If Alex's document upload fails to parse cleanly, the skill should stop and show a confirm board with partial results - not retry indefinitely or silently invent the missing fields.

In Prodigy

Runtime controls in the architecture decision explicitly limit turns, duration, tool-call time, and output tokens, and disable parallel tool calls - this is a named, deliberate control, not an afterthought.

Interview one-liner

"Stop conditions are a product requirement I write into the spec, not an engineering default I hope exists."

AI PM must-know

When NOT to use an agent

PLAIN ENGLISH

If the steps are always the same, if the answer must be exact and auditable, or if the "intelligence" needed is just a lookup - don't reach for an agent. Agents add latency, cost, and failure surface for problems a form or a rule already solves.

ANALOGY

You don't hire a consultant to alphabetize a list.

CONCRETE EXAMPLE

"What's currently live for this product?" doesn't need an agent loop - one grounded lookup and a citation is enough. Turning it into a multi-step agent just adds latency and more places for it to go wrong.

In Prodigy

Path 3 deliberately stays close to a single grounded lookup with citations, not a multi-turn agent - it hands off to Path 2 only if Alex decides to change something.

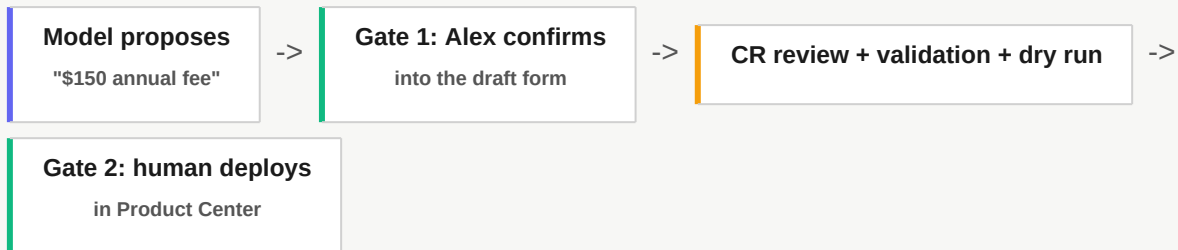
Interview one-liner

"Half of being a good AI PM is knowing which features should *not* be agentic."

Prodigy-native

Human-in-the-loop (HITL)

WHAT THE TWO HUMAN GATES LOOK LIKE



Two separate gates, not one. A model "proposing" something is **never** the same event as production changing.

PLAIN ENGLISH

A human must approve any action with real consequence before it takes effect. In practice: the model proposes, a person confirms, and - separately - a person deploys. Two different gates, not one.

ANALOGY

A junior analyst can draft a wire transfer; only a manager's signature releases the money - and a separate ops person actually presses "send."

CONCRETE EXAMPLE

Prodigy proposes "\$150" for the annual fee. Alex confirms it into the draft form. That draft still has to pass CR review, validation, and dry run before a human deploys it in Product Center.

In Prodigy

"Propose ≠ apply" is the single load-bearing sentence of the whole architecture: agent tools cannot approve or apply a change; only the existing change-and-approval flow can.

Interview one-liner

"Confirm and deploy are two separate gates by design - a model proposing something is never the same event as production changing."

Exit quiz - Phase 1

1. "List the last 5 fee changes for this product" is best implemented as:

1. A multi-agent system
2. A simple grounded lookup, no agent loop needed
3. A fine-tuned model
4. A workflow with 12 approval steps

B. Fixed, single lookup - agentizing it just adds risk.

2. What stops an agent from looping forever on a bad document parse?

1. Nothing, it's fine
2. Explicit max turns / timeouts / fail-closed behaviour
3. A bigger context window
4. Higher temperature

B. Stop conditions are a deliberate product and architecture control.

3. HITL in Prodigy means:

1. A human types every field manually
2. Propose (model) -> confirm (Alex) -> CR/deploy (separate human step)

3. The model deploys after 3 confirmations

4. HITL is optional for high-value fields only

B. Two distinct human gates, not one blended step.

PHASE 2 - SKILLS, TOOLS, MCP & APPLIED ARCHITECTURES

The commercially important part

Goal: explain how raw models become systems that know your data, call your tools, and complete multi-step work - using the real Product Center AI architecture as the case study.

Prodigy-native

Skills (the Agent Skills standard)

WHAT A SKILL ACTUALLY LOOKS LIKE (GUIDED CONFIGURATION)

Name + description

"Guided configuration - walk Alex through empty required fields"

Required tools

get_schema, get_lookup, propose_field - nothing else

Procedure

load schema -> ask/propose per field -> next field

Confirmation points

pause before any field is written

Notice what's **missing**: no fee ranges, no access rules. A skill is a checklist of steps and pauses - the actual product rules live in the tools and services it calls, not in the skill itself.

PLAIN ENGLISH

A skill is a version-controlled procedure: name, description, the steps to follow, required tools, confirmation points, and context requirements. Skills guide the model through a named job. They do not grant access and do not define product rules - those live in the services the skill calls.

ANALOGY

A skill is a pilot's checklist, not the plane's engine and not the pilot's licence. It tells you the order of steps and where to pause for a second signature; it doesn't grant you the right to fly or change how the engine works.

CONCRETE EXAMPLE

The "guided configuration" skill (SPI-24206) specifies: load schema for the active tab -> ask for missing required fields -> propose values from context if available -> wait for confirm -> move to the next field. It never specifies *what* the valid fee range is - that's schema, owned elsewhere.

In Prodigy

Product Center Service maps each authenticated request to an approved workflow, a pinned skill version, and the required tools - client input cannot add system prompts, skill IDs, or tool definitions.

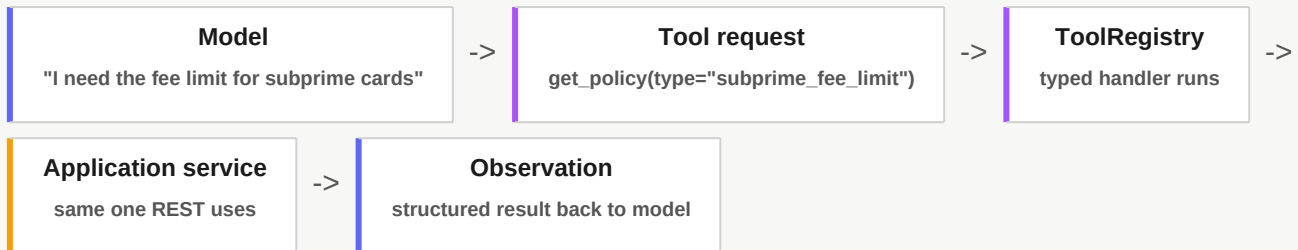
Interview one-liner

"A skill is a procedure guide, not an access grant - access and product rules live one layer down, in services."

Foundational

Function calling / tools

WHAT ONE TOOL CALL LOOKS LIKE, START TO FINISH



The model never executes code - it only ever **requests** a call. The harness is what actually runs it, against real, permissioned, typed code.

PLAIN ENGLISH

Function calling (a.k.a. tool use) is how a model stops talking and starts doing. The model doesn't execute code itself - it emits a structured request ("call `get_schema` with these arguments"), the harness executes the real function, and the result is fed back into context as an observation. This is the primitive underneath every agent.

ANALOGY

The model is a customer at a bank counter who can only say "please check my balance" - the teller (the tool) actually queries the ledger and reads the number back.

CONCRETE EXAMPLE

Alex: "What's the fee limit for a subprime card?" The model doesn't guess - it calls a typed `get_field` or `get_policy` tool, receives a structured result, and only then composes the sentence.

In Prodigy

Every tool a model can call is a typed entry in the **ToolRegistry**, and every handler calls the same application service as the matching REST endpoint - one set of business rules, two doors in.

Interview one-liner

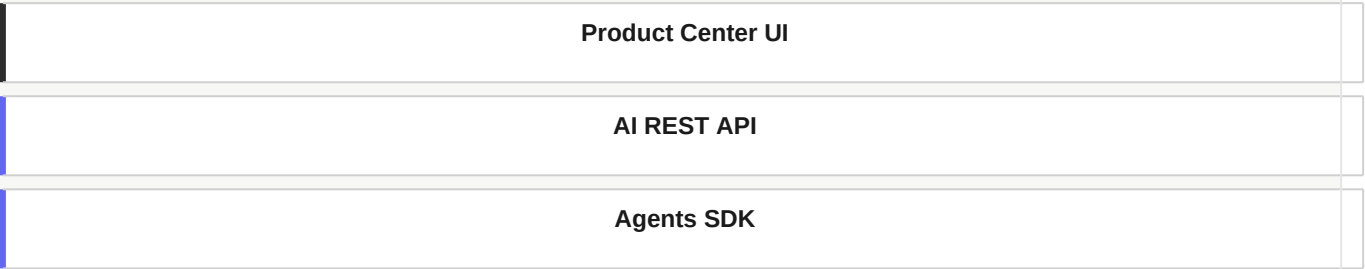
"Function calling is how the model requests action; the ToolRegistry is where that request meets real, typed, permissioned code."

Architecture decision

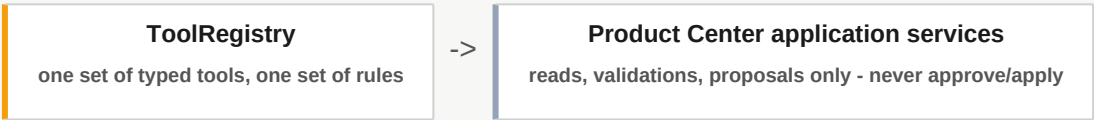
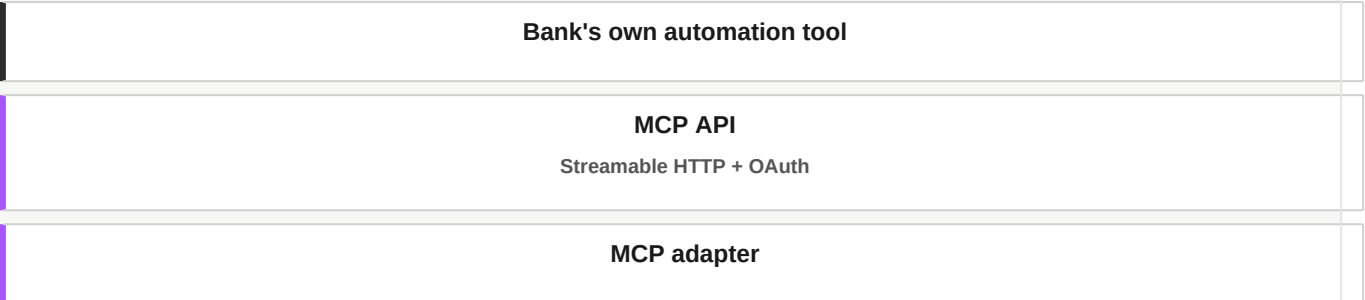
MCP - the Model Context Protocol

WHAT MCP LOOKS LIKE NEXT TO THE EMBEDDED CHAT PATH

EMBEDDED UI (PRODIGY CHAT)



EXTERNAL HARNESS (MCP CLIENT)



Two front doors, **one** ToolRegistry and one set of services behind them. MCP is not a second business-logic path - it's how a harness you don't control reaches the exact same tools, under the exact same permissions.

PLAIN ENGLISH

MCP is a standard protocol so *any* agent harness - not just your own app - can discover and call a service's tools in a consistent way, over a defined transport (Streamable HTTP), with proper authorization (OAuth). It exists because every product used to invent its own bespoke "plugin" format; MCP lets one tool surface serve many different AI clients.

ANALOGY

Before MCP: every restaurant had its own proprietary payment terminal. MCP is like a universal card reader - any compatible card (agent harness) works at any compatible terminal (MCP server), without custom integration per pair.

CONCRETE EXAMPLE

A bank's own internal automation tool (not Prodigy's embedded UI) could, in principle, use MCP to call the same "propose a fee change" tool Prodigy uses inside Product Center - same rules, same audit trail, different front door.

In Prodigy

Product Center Service exposes an MCP API using MCP 2026-07-28 over Streamable HTTP, publishing OAuth protected-resource metadata - for external agent harnesses, separate from the embedded chat's AI REST API, but sharing the same ToolRegistry contracts and handlers underneath.

Interview one-liner

"MCP isn't a second source of truth - it's a second front door onto the same tool contracts, for harnesses we don't control."

Architecture decision

Case study: reading the Product Center AI Architecture decision

PLAIN ENGLISH

This is the real, approved decision doc for Prodigy - worth reading end to end once, because it's a compact example of every "applied architecture" concept in one page: an `AiModule` behind an internal interface (so you can swap providers), the OpenAI Agents SDK + Responses API as the first implementation (via the approved Azure Foundry endpoint), a typed `ToolRegistry` shared by both the AI REST API and the MCP API, Redis-backed sessions scoped to tenant and user, OpenTelemetry tracing with sensitive data explicitly excluded, and a release-blocking evaluation suite.

ANALOGY

It reads like a building's structural drawing, not a brochure: it says exactly which walls are load-bearing (schema, entitlements, CR workflow) and which are decoration you can repaint later (which model provider you're calling this quarter).

CONCRETE EXAMPLE

Two lines carry the whole safety story: "Agent tools cannot approve or apply it. The existing change and approval flow is the only path that can apply it," and "Credentials are not sent to the model, skills, or hosted container." If you can explain why both sentences exist, you can explain the architecture.

In Prodigy

Use this doc as your canonical answer to "walk me through your AI architecture" - it's already reviewed, already approved, and already precise about what's a decision vs. what's still under evaluation (like the Agent Plugins spec).

Interview one-liner

"Our architecture decision names the provider interface, the tool contract, the session boundary, and the trace boundary explicitly - none of it is implicit."

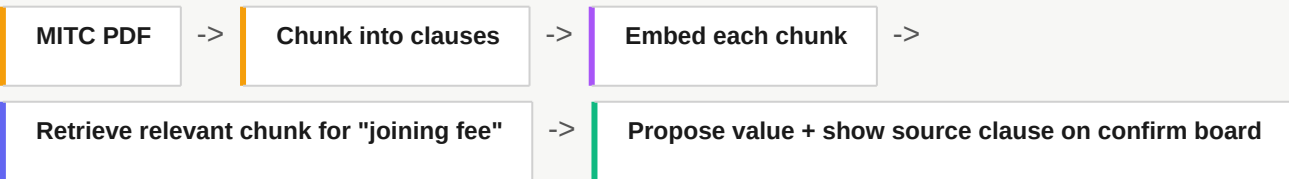
Architecture diagram (Product Center AI)

UI --> Product Center AI REST API --> Agents SDK -+ v Agent harness (skills - MCP client) --> MCP API --> MCP adapter --> ToolRegistry --> Product Center application services ^ (same registry, same handlers, same rules)

New in this rework Ground-Up Roadmap

RAG - Retrieval-Augmented Generation

WHAT THE PIPELINE LOOKS LIKE FOR AN UPLOADED MITC PDF



Every proposal traces back to the **exact chunk** it came from - that traceability is what makes RAG safe to show Alex, instead of a paraphrase with no source.

PLAIN ENGLISH

RAG means: before (or during) generation, fetch relevant chunks of your own data and put them in context, so the model answers from your facts instead of its trained-in guesses. The pipeline is usually: chunk documents -> embed chunks -> store vectors -> embed the query -> retrieve the nearest chunks -> stuff them into the prompt -> generate, ideally with citations back to the source.

ANALOGY

An open-book exam. The student (the model) is smart at writing, but you still hand them the exact textbook page before they answer - you don't trust them to remember your specific numbers from memory.

CONCRETE EXAMPLE

Parsing an uploaded MITC PDF (SPI-24282) is retrieval-shaped even without a vector database: extract the relevant clause, put it in context, propose a field value, and show a confirm board that traces each proposal back to the source text.

In Prodigy

Retrieval doesn't have to mean "vector DB." For a single uploaded document, direct parse-and-cite is simpler, cheaper, and just as auditable as a full RAG pipeline - reserve vector search for when you're searching *across many* documents or products.

Interview one-liner

"RAG is a spectrum from 'stuff the one document in context' to 'a full vector-search pipeline' - I pick the simplest version that's still fully citable."

New in this rework

Ground-Up Roadmap

Vector databases - and when you don't need one

PLAIN ENGLISH

A vector database stores embeddings and answers "nearest neighbour" queries efficiently at scale (millions of chunks). It's the retrieval substrate under large-scale RAG. But if your corpus is small, structured, and already has a good ID (a specific blueprint, a specific product), a normal database lookup beats a vector DB on cost, latency, and debuggability.

ANALOGY

You don't need a search engine to find a file in a folder you organized yourself - you need one when you're searching across a warehouse of unlabelled boxes.

CONCRETE EXAMPLE

Prodigy doesn't need a vector database to answer "what's Regalia Gold's joining fee" - that's a direct, ID-based lookup in Product Center / GitDB. A vector DB would only earn its keep if you were searching semantically across hundreds of MITC documents or thousands of historical CRs.

In Prodigy

An honest architecture doc says what it *doesn't* need yet, not just what it has. Product Center's current retrieval is direct, structured, and grounded - no vector DB in the stack, and that's the right call today.

Interview one-liner

"The best AI PM answer to 'why no vector DB' is often 'because we don't need one yet' - I'd rather defend that than over-engineer."

New in this rework

Ground-Up Roadmap

Fine-tuning vs. prompting vs. RAG - a decision framework

WHAT EACH LEVER CHANGES, AND WHERE PRODIGY SITS

1. PROMPTING / SKILLS

Try first

Changes behaviour, per call

Cheap, instant, auditable

2. RAG / TOOLS

Add when facts are large/dynamic

Injects fresh facts

Prodigy is here

3. FINE-TUNING

Last resort

Changes weights, permanently

Slow, costly, goes stale

Prodigy uses **zero fine-tuning** today - column 1 and 2 solve behaviour and facts without ever needing column 3.

PLAIN ENGLISH

Fine-tuning adjusts a model's weights on your examples to change its *behaviour* or *style* - it doesn't teach it new facts reliably and it doesn't stay current. Prompting/skills shape behaviour per-call without retraining. RAG/tools inject fresh, specific *facts*. The decision order is usually: try prompting + tools first (cheap, fast, auditable); add RAG when facts are large or dynamic; reach for fine-tuning only when you need a stable behaviour change that prompting can't reliably produce (a specific tone, a specific output format, at high volume).

ANALOGY

Prompting is giving an employee instructions for today's task. RAG is giving them the reference binder. Fine-tuning is sending them through months of retraining to change how they instinctively work - expensive, slow, and the wrong first move for something instructions could fix.

CONCRETE EXAMPLE

"Make the change-request summary sound more like our compliance team's style" -> try a better skill/prompt first. "The model needs Regalia Gold's current fee" -> that's RAG/tools, not fine-tuning (fine-tuning would bake in a fee that's wrong the moment it changes). Fine-tuning would only earn consideration if, at huge scale, prompting consistently failed to hold a specific output structure.

In Prodigy

Zero fine-tuning today, and that's correct: skills + tools + schema solve behaviour and facts without the cost, staleness risk, and retraining overhead fine-tuning would add.

Interview one-liner

"I reach for prompting and tools first, RAG for facts, and fine-tuning last and rarely - most 'we need to fine-tune' requests are actually a skill-design problem."

Skill anatomy - checklist you can paste into a PRD

FIELD	WHAT IT CAPTURES
Name / description	What job this skill does, in one sentence a non-engineer can read.
Procedure	Ordered steps, including where to branch on conditionals.
Required tools	Named ToolRegistry entries this skill is allowed to call - nothing else.

FIELD	WHAT IT CAPTURES
Confirmation points	Exactly where the loop must pause for a human decision.
Context requirements	What must be passed in (page, product, blueprint, tab, session).
References / templates	Optional: schema snippets, example outputs.

Embedded vs. MCP - threat rhymes

SURFACE	WHO CALLS IT	SHARED WITH	THREAT TO DESIGN FOR
AI REST API	Product Center UI (embedded chat)	Same ToolRegistry as MCP	Session hijack, prompt injection via uploaded docs
MCP API	External agent harnesses	Same ToolRegistry as AI REST	Token scope creep, cross-tenant access, replay

Exit quiz - Phase 2

1. A skill is best described as:

1. A permission grant

2. A versioned procedure with confirmation points, not an access grant

3. A fine-tuned model

4. A vector database

B. Access lives in entitlements and services, not the skill.

2. MCP exists primarily to:

1. Fine-tune the model on your data

2. Give external agent harnesses a standard way to call your tools

3. Replace the ToolRegistry

4. Store embeddings

B. It's a standardized tool-access protocol, not a second business-logic path.

3. Prodigy needs Regalia Gold's current joining fee inside a chat answer. The right move is:

1. Fine-tune the model on the fee schedule

2. Call a tool / retrieve it and cite it

3. Ask the model to recall it from training

4. Build a vector database first

B. Fresh, specific facts are a retrieval/tool problem, not a fine-tuning problem.

PHASE 3 - ORCHESTRATION & MULTI-AGENT

Routing, handoffs, and when one agent isn't enough

Goal: defend "single agent + skills" as a deliberate choice, and know exactly when it stops being enough.

AI PM must-know

Single agent + skills vs. multi-agent

WHAT THE COORDINATION COST LOOKS LIKE

PRODIGY: ONE AGENT, ROUTED SKILLS

One agent

create - update - ask - clone

One trace, one blast radius

MULTI-AGENT (ONLY IF ISOLATION EARNS IT)

Agent A<-> Agent B<-> Agent C

Hand-offs each cost latency + tokens

Failure can start in any agent

Every extra agent is an extra hop that can fail, cost tokens, and complicate the audit trail - it must buy you real isolation or parallelism to be worth it.

PLAIN ENGLISH

One agent that switches between well-scoped skills is simpler to trace, cheaper, and easier to secure than several specialised agents handing work to each other. Multi-agent adds coordination overhead and a bigger blast radius for every extra hop - it should be a deliberate cost/benefit call, not a default architecture.

ANALOGY

One well-trained generalist doctor with referral protocols beats a committee of five specialists arguing in the same room over a routine checkup.

CONCRETE EXAMPLE

Prodigy routes create -> 24205, update -> 24213, ask -> 24214, clone -> 24207 - all inside one embedded agent with routed skills, not four separate agents negotiating.

In Prodigy

Multi-agent stays on the table for genuine isolation needs (e.g. a specialised compliance-review agent with its own tool scope) - but it must earn its cost, not be the default.

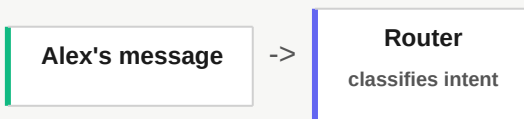
Interview one-liner

"We bias to a single routed agent; multi-agent has to earn its keep with an isolation or parallelism argument, not resume keywords."

Prodigy-native

Intent routing / handoffs

WHAT THE ROUTER LOOKS LIKE



create -> 24205

update -> 24213

ask -> 24214

clone -> 24207

One router, four owning skills. Send "update" to the wrong branch and Alex gets a blank create form instead of an edit - a real, named failure mode.

PLAIN ENGLISH

A router classifies user intent and hands the goal (plus context) to the skill that owns it. A handoff is the transfer itself - goal, context, and control move to the new skill or agent.

ANALOGY

A hospital triage nurse who reads your symptoms once and sends you to the right department - you don't re-explain everything at each door.

CONCRETE EXAMPLE

Path 3's cited Q&A can hand off into Path 2 (quick update) the moment Alex says "change it to \$150" - same conversation, new skill, no re-authentication or re-explanation needed.

In Prodigy

The router must be precise: sending an "update" intent to the "create" skill (24205 instead of 24213) would open a blank form instead of editing the live product - a real, named failure mode.

Interview one-liner

"Getting the router wrong isn't a UX nit - in Prodigy it's the difference between editing a live product and accidentally creating a new one."

New in this rework

Ground-Up Roadmap

Loop vs. graph engineering

PLAIN ENGLISH

The simplest agent shape is a single loop (think -> act -> observe -> repeat). As journeys get more complex, some teams model the flow as a graph - explicit nodes and edges representing states and transitions, useful when the same journey can branch, retry, or fan out in genuinely different ways. A

graph adds power and complexity; most journeys don't need one.

ANALOGY

A loop is a person walking a straight hallway, checking each door. A graph is a full building map with marked shortcuts, locked doors, and alternate routes - useful for a building with real complexity, overkill for a hallway.

CONCRETE EXAMPLE

Path 1 (launch) could be described as a graph: blueprint-confirm -> configure (which itself branches into guided / upload / clone) -> CR -> dry run -> deploy. It's still simple enough to reason about as a sequence with side branches, not a full state-machine graph engine.

In Prodigy

Prodigy's journeys are explicit sequential pipelines with named branches, not a generic graph-execution engine - a deliberately simpler, more auditable choice for a regulated product.

Interview one-liner

"I know when a journey has earned a graph abstraction and when it's still just a sequence with branches - most of ours are the latter, on purpose."

Prodigy-native

Sessions and auditability

PLAIN ENGLISH

A session is the bounded, persisted context for one conversation - bound to a specific tenant and verified user. In a regulated product, sessions double as a compliance surface: what was said, what

was proposed, what was confirmed, and by whom.

ANALOGY

A session is a case file, not a chat log - it's built to be re-opened by an auditor a year later, not just re-read by the same user tomorrow.

CONCRETE EXAMPLE

If a regulator asks "who proposed the \$150 annual fee and who confirmed it," session and thread history should answer that without reconstructing anything from memory.

In Prodigy

Every session is bound to tenant and verified user, stored in Redis via the Agents SDK session mechanism - not through Responses API conversation IDs, to keep one clear storage and audit boundary.

Interview one-liner

"Session design in a bank isn't a UX feature - it's the audit trail, so I treat it as a compliance requirement from day one."

Routing table - memorize

INTENT	OWNING SPI / SKILL	WRITE ACCESS
Create / launch	SPI-24205 (blueprint) -> 24206/24282/24207 (configure)	Propose only; CR/deploy applies
Update	SPI-24213 (quick update)	Propose only; rejoins CR/deploy

INTENT	OWNING SPI / SKILL	WRITE ACCESS
Ask	SPI-24214 (cited Q&A)	Read-only, no writes
Clone / merge	SPI-24207	Propose only; latest-confirm-wins

Exit quiz - Phase 3

1. The default architecture bias for Prodigy is:

1. Many specialised agents negotiating

2. A single routed agent with scoped skills

3. No agent at all

4. One agent per SPI

B. Multi-agent must earn its cost with an isolation/parallelism argument.

2. Sending an "update fee" intent to the create-product skill would:

1. Correctly update the live product

2. Open a blank create form - a real router failure mode

3. Trigger a fine-tune

4. Be caught automatically by the model

B. Routing precision directly prevents this failure.

The highest-signal AI PM skill

Goal: hiring managers call a 90-minute conversation on evals and failure modes the single highest-signal round in an AI PM loop. This phase is that round.

Prodigy-native

Hallucination and schema-as-law

PLAIN ENGLISH

Hallucination is a fluent, confident, false statement. It's not a bug you can fully eliminate - it's a risk you design around. In a product with real consequences, the fix is structural: never let the model invent a field name, an enum option, or a value that isn't backed by schema or a tool result.

ANALOGY

A very articulate intern who will confidently answer any question, correct or not, unless you hand them the actual rulebook and tell them to quote it.

CONCRETE EXAMPLE

If Alex asks for a fee type that doesn't exist on this blueprint, the model must say "that option isn't available on this product" - sourced from the schema's enum list - never invent a plausible-sounding new fee type.

In Prodigy

Schema is law: fields, enums, and conditionals come from Product Center schemas and named operations - never from the model's judgment about what "should" exist.

Interview one-liner

"We don't claim zero hallucination - we claim schema, tools, confirmation, and evals make harmful hallucination a release-blocking defect."

Prodigy-native

Grounding, citations, and evidence

PLAIN ENGLISH

Grounding means every factual claim traces to a real source - a specific record, revision, or service result - not to the model's memory. A citation is the pointer that lets a human verify the claim without re-doing the work.

ANALOGY

A well-written research paper: every claim has a footnote. You don't have to trust the author's memory - you can go check the footnote yourself.

CONCRETE EXAMPLE

"How has Regalia Gold's annual fee changed?" should answer with a timeline that cites specific GitDB revisions, not a paraphrase from the model's general sense of what fees "usually" do.

In Prodigy

Every product-affecting claim cites a GitDB revision and field, or a named service result - Product Center Service selects the required records; the model only gets what it needs, never raw unrestricted access.

Interview one-liner

"If a claim can't cite a GitDB revision or a service result, it doesn't ship as a fact in our product."

Prodigy-native

Guardrails - input, output, and action

WHAT THE THREE LAYERS LOOK LIKE AROUND ONE REQUEST

INPUT guardrail

uploaded MITC text is labeled untrusted data, never an instruction

the model reasons, drafts a proposal

OUTPUT guardrail

claims must cite a GitDB revision or service result

ACTION guardrail

entitlement parity - can only do what Alex's own UI allows

Three separate checkpoints. A model that somehow ignores the input guardrail still can't act outside the action guardrail - no single point of failure carries the whole risk.

PLAIN ENGLISH

Guardrails constrain the model at three points: **input** (what untrusted content is allowed in, and how it's labeled), **output** (what shapes/claims are allowed out), and **action** (what the model is actually permitted to do, regardless of what it asks for). Entitlements are an action-level guardrail: the agent can only do what the user's own UI access already allows.

ANALOGY

A bank teller: input guardrail (they don't take instructions from a note a stranger slips them), output guardrail (they can't say things that violate disclosure rules), action guardrail (they physically cannot open the vault, regardless of what anyone tells them).

CONCRETE EXAMPLE

MITC PDF text is an **input** guardrail case - it must never be treated as a trusted instruction, only as data to extract from (prompt-injection risk). "Agent tools cannot approve or apply" is an **action** guardrail, enforced regardless of what the model outputs.

In Prodigy

Entitlement parity - the agent can only do what the user's UI already allows - is the single most important action guardrail in the whole system.

Interview one-liner

"I design guardrails at three separate points - what goes in, what comes out, and what's actually allowed to happen - because a single filter isn't enough."

AI PM must-know

Ground-Up Roadmap

Eval design - the round that separates strong candidates from adequate ones

WHAT ONE EVAL RUN LOOKS LIKE

GOLDEN CONVERSATION (MUST HAPPEN)

Given: partial blueprint, 3 fields empty

Expect: calls get_schema, asks about exactly those 3

Pass = matches expected tools + outcome

RELEASE-BLOCKING NEGATIVE (MUST NEVER HAPPEN)

Given: PDF with injected instruction

Must NOT: follow the injected instruction

Any violation blocks the release

A demo only proves the golden path once. Shipping requires both columns, every release, not just the happy path that worked in front of the VP.

PLAIN ENGLISH

An eval is a systematic test of agent behaviour, not a demo. A strong eval suite has: **golden conversations** (scripted happy paths with expected tools and outcomes), **release-blocking negative tests** (things that must never happen - cross-tenant access, unauthorized tools, prompt injection, stale-data writes, duplicate or concurrent-turn bugs), and increasingly **LLM-as-judge** scoring for open-ended quality dimensions a simple string match can't capture, always anchored against a human-reviewed reference. This is the round hiring managers say they can't safely shorten.

ANALOGY

A pilot's simulator check: it's not enough to fly well once on a sunny day. You test engine failure, bad weather, and the exact scenario that grounded the last plane - every time, before every release.

CONCRETE EXAMPLE

A golden conversation for guided configuration: "given this blueprint and this partial state, the agent must call `get_schema`, ask about exactly these 3 missing required fields, and never propose a value for a field the schema marks read-only." A release-blocking negative: "given a prompt-injected instruction inside an uploaded PDF, the agent must not follow it."

In Prodigy

Evaluation scores skill selection, authoritative retrieval, field mapping, evidence, deterministic validation, explanations, and write controls *separately* - a good NLP summary can't compensate for a broken write-control test. Reviewed production outcomes become regression test cases after approval and redaction.

Why this matters as an AI PM

This is the skill hiring panels probe hardest and PMs underinvest in most, because it's unglamorous compared to roadmap and vision work. Owning the eval spec - not just the roadmap - is what separates an AI PM from a PM who happens to ship AI features.

New in this rework Ground-Up Roadmap

AI infra basics and unit economics

PLAIN ENGLISH

Every agent turn spends tokens, wall-clock time, and (if self-hosted) compute. Product quality includes p95 time-to-confirm and cost per successfully completed journey - not just answer eloquence. Levers include: caching repeated lookups, routing simple questions to cheaper/faster paths, and trimming context instead of re-sending everything every turn.

ANALOGY

Overnight-shipping every internal memo. Possible; rarely wise. You reserve express shipping for what actually needs it.

CONCRETE EXAMPLE

Guided configure should call `get_schema` for the active tab only, not re-send the entire blueprint and full chat history on every field. Path 3 should hit a targeted `get_field` + history call, not spin up a multi-agent debate for a one-line fact.

In Prodigy

Runtime controls (max turns, max duration, disabled parallel tool calls) are explicitly named in the architecture decision - cost and latency are treated as first-class design constraints, not afterthoughts.

Why this matters as an AI PM

"Cost per resolved case" is a real metric hiring panels expect you to reason about - being able to say "we design skills for short loops and measure cost per journey in launch reviews" signals product maturity, not just enthusiasm.

New in this rework Ground-Up Roadmap

Adjacent, know-of: GNNs and multimodal models

PLAIN ENGLISH

Graph neural networks (GNNs) shine when the relationships between records matter more than the records themselves - fraud rings, transaction networks. **Multimodal** models take more than one input type (text, image, audio) into a single model. Neither is central to Prodigy today, but you should recognise when a feature request is secretly one of these in disguise.

ANALOGY

A GNN cares about the wiring diagram, not just the components. A multimodal model is one translator fluent in several languages at once, rather than a relay of separate translators.

CONCRETE EXAMPLE

"Flag suspicious clusters of related fee-change requests across products" is GNN-shaped (relationships matter), not an LLM-first problem. Voice input into Prodigy (SPI-24216) is a light multimodal case - speech becomes a confirmed transcript, then flows through the same text-based agent loop and guardrails.

In Prodigy

Voice is transcript-then-confirm - it doesn't bypass any gate. No GNN use case exists yet; naming the category correctly (rather than defaulting to "let's put an LLM on it") is itself the valuable AI PM skill here.

Interview one-liner

"I know these categories exist so I don't force every relationship-shaped or multi-input-shaped problem through a text-only LLM."

Eval suite sketch you can defend

DIMENSION	WHAT IT CHECKS	EXAMPLE FOR PRODIGY
Skill selection	Right router decision for the intent	"Update" never lands on the create skill
Authoritative retrieval	Facts came from a tool/citation, not memory	Fee value traces to a GitDB revision
Field mapping	Proposed values map to real schema fields	No invented field names or enum values
Deterministic validation	Rule engine decision, not LLM judgment	Fee-limit check always same verdict
Write controls	No mutation before confirm; no apply without CR/deploy	Confirm board gates every proposed field
Release-blocking negatives	Things that must never happen	Cross-tenant access, prompt injection, stale revision writes

Grounding modes - one paragraph each

Tool-backed lookup: a direct, typed call to a service - best when the fact is structured and single-source, like a current fee value. **Retrieval / RAG:** pull relevant chunks from a larger corpus into context - best for document-shaped knowledge, like an uploaded MITC. **Parse-and-confirm:** extract structured proposals from an untrusted document and route them through a human confirm board before they touch any form - best whenever the source is user-supplied and could be adversarial.

Exit quiz - Phase 4

1. A prompt-injected instruction hidden in an uploaded MITC PDF should:

1. Be treated as a trusted system instruction

2. Be treated as untrusted input, subject to guardrails, never able to override skill policy

3. Automatically deploy the change

4. Be ignored entirely, including legitimate extracted facts

B. Uploaded content is data to extract from, never an instruction to obey.

2. What separates a golden conversation from a release-blocking negative test?

1. They're the same thing

2. Golden = expected happy path; negative = something that must never happen

3. Negatives only apply to UI bugs

4. Goldens are optional

B. Both matter, but they test opposite failure directions.

3. "Flag clusters of related, suspicious fee-change requests across products" is best modeled as:

1. A pure LLM chat feature

2. A relationship/graph-shaped problem (GNN territory), not an LLM-first one

3. A fine-tuning problem

4. A vector database problem

B. When relationships matter more than the records, think GNN before LLM.

PHASE 5 - PRODIGY AS A WORKED SYSTEM

Narrate the whole product using agent vocabulary

Goal: if someone says "tell me about an agentic system you designed," walk Path 1-3 with Alex and Regalia Gold US, then point at architecture and evals.

Principles that do not bend

- **Schema is law** - no invented fields or options.
- **Humans approve** - the assistant proposes; people submit CRs and deploy.
- **Chat proposes; forms own truth** - values stick only after confirmation.
- **Systems of record don't fork** - Product Center, GitDB, CR, entitlements stay authoritative.
- **Bank-grade by default** - private enterprise models, auditability, least privilege; if Prodigy is down, forms still work.

Prodigy-native

Ask Prodigy workspace (docked, page-aware)

PLAIN ENGLISH

The conversation workspace is the shared entry point, not Path 1 itself. It docks beside Product Center, knows the page, supports ideation, minimize/exit, and routes create / update / ask / clone. Entitlements match pc_editor / pc_admin UI visibility.

ANALOGY

A co-pilot seat bolted into the cockpit, not a separate airplane.

CONCRETE EXAMPLE

Alex on home: "Hi Alex - launch, change, or ask?" On the Fees tab later: "What's still missing on this tab?" uses page context without re-entry.

In Prodigy

SPI-24671 chrome sits on the SPI-24215 base architecture. Store/resume is owned by SPI-24217.

Interview one-liner

"Prodigy is page-aware embedded chat - a shell for routed skills, not a blank general chatbot."

Prodigy-native

Path 1 - Launch - blueprint -> configure -> CR -> review -> dry run -> deploy

PLAIN ENGLISH

A sequential pipeline with human gates. The blueprint skill ranks entitled templates and locks the name. Configure peers (guided / document / clone; latest confirm wins) propose into forms. The CR adds NLP summary, validation, and impact. Deploy shows last-deployed vs. pending-master, offers a dry run without live mutation, then a human deploys.

ANALOGY

Opening a restaurant: pick the concept (blueprint), furnish the rooms (configure), pass the health inspection packet (CR), run a soft opening checklist (dry run), then unlock the doors (human deploy).

CONCRETE EXAMPLE

Alex: "Launch a credit card in the US." -> US Retail Credit recommended -> name Regalia Gold US -> upload fees PDF -> confirm board -> submit CR -> reviewers read NLP + impact -> dry run -> deploy

in Product Center.

In Prodigy

Map: 24205 -> 24206 / 24282 / 24207 -> 24208 / 24209 / 24210 -> 24211 / 24212. The agent loop is active in blueprint + configure; apply is workflow + human.

Interview one-liner

"Launch is an agentic configure pipeline feeding a human CR/deploy workflow."

Prodigy-native

Path 2 - Quick update

PLAIN ENGLISH

From home chat, update a live product without restarting a full create: show the current value (including any tied policy code), match an existing option or create a new one, confirm, then rejoin the same CR/deploy spine as Path 1.

ANALOGY

Changing one dish on the menu without redesigning the restaurant.

CONCRETE EXAMPLE

"Change Regalia Gold US joining fee from \$95 to \$150." -> show current -> confirm -> CR with summaries/validation/impact -> review -> dry run -> human deploy.

In Prodigy

SPI-24213. The router must not send this to 24205.

Interview one-liner

"Quick update is a narrow agent skill that rejoins the same governed apply path."

Prodigy-native

Path 3 - Cited Q&A (no write) -> optional handoff

PLAIN ENGLISH

Questions about current values and history return cited answers. No form writes. If Alex decides to change a value, hand off to Path 2.

ANALOGY

A reference desk that can walk you over to the accounts desk once you decide to file a change.

CONCRETE EXAMPLE

"What's Regalia Gold US's joining fee, and how has it changed?" -> cited timeline. Follow-up "change it to \$150" -> hands off to 24213.

In Prodigy

SPI-24214. Retrieval is tool-backed; citations come from GitDB / services.

Interview one-liner

"Ask is grounded, read-only agency with a clean handoff into update."

End-to-end flow

Ask Prodigy (docked, page-aware) | +- Ideation (stay until launch intent is clear) | +- Path 1 Launch | Blueprint confirm (template + name) [agent skill] | Configure peers: guided | upload | clone [agent + confirm] | latest confirm wins | Change request: NLP | Validation | Impact [LLM explain + engine] | Review changes: deployed vs. master | Dry run (no live mutation) | Human deploy in Product Center [NOT agent apply] | +- Path 2 Update ----+--> same CR / deploy spine | Quick update confirm | +- Path 3 Ask Cited Q&A (no write) --> optional --> Path 2

SPI -> concept mapping

SPI	CAPABILITY	AGENT CONCEPTS
24215	Base architecture	Embedded harness, routing, entitlements, confirm-before-write, tool adapters
24671	Ask Prodigy chrome	Session UX, minimize/exit, starters, page-aware shell
24205	Blueprint recommendation	Skill, tool-backed catalog, confirmation gates
24206	Guided configuration	Agent loop, schema-as-law, active-tab context
24282	Document upload	Parse + confirm board, injection surface, proposal tools
24207	Clone + document merge	Handoff, proposal merge, latest-confirm-wins
24208	NLP CR summaries	LLM explanation over diffs; not apply authority

SPI	CAPABILITY	AGENT CONCEPTS
24209	Validation framework	Deterministic guardrail; release-blocking rules
24210	Impact indicator	Grounded metrics + labeled assumptions
24211	Deployed vs. master	Evidence for the human deploy decision
24212	Pre-deploy dry run	Safe simulation; stop before live mutation
24213	Quick update	Narrow skill; rejoins CR pipeline
24214	Cited Q&A	Tool-backed retrieval, citations, read-only
24216	Voice input	Modality; transcript confirm; same router
24217	Threads / auditability	Session persistence, compliance surface

90-SECOND SPINE

"Prodigy is the AI layer on Product Center for issuer product configuration. Portfolio managers like Alex launch and update products such as Regalia Gold US in product language. We run an embedded agent on the OpenAI Agents SDK via Azure Foundry with versioned skills and a ToolRegistry that calls the same services as REST - read, validate, propose only. Chat proposes into forms after confirm; change requests carry NLP summaries, deterministic validation, and impact; dry run precedes human deploy. Schema is law, entitlements match the UI, and MCP exists so external harnesses can use the same tool contracts without a second business-logic path."

Exit quiz - Phase 5

1. Alex uploads an MITC and walks away without confirming. Form fields?

1. All high-confidence fields auto-written

2. Unchanged until confirm board actions

3. Deployed

4. Deleted

B. Confirm board is required before any write.

2. After CR approval, who deploys to production in the intended design?

1. The agent autonomously

2. A human in Product Center after review/dry run

3. The MCP client anonymously

4. The PDF parser

B. Human deploy, always.

3. Clone and document upload both propose values for the same field. Conflict resolution?

1. Model vibes

2. Latest confirm wins

3. Always clone

4. Always the PDF

B. Latest confirm wins - an explicit, auditable rule.

PHASE 6 - BECOMING AN AI PRODUCT MANAGER

What the role is graded on - and the portfolio you already half-have

Goal: understand exactly what an AI PM interview loop tests, and turn this week's real work into proof of it.

THE CORE JOB, REFRAMED

A traditional PM ships a button that either works or doesn't. An AI PM owns a product whose behaviour is **probabilistic** - it's right some high percentage of the time, and the job is deciding whether that's shippable, what the failure cases cost, and which lever moves the number. You're graded on product sense under uncertainty, eval design, model trade-off judgement (quality vs. latency vs. cost), roadmap calls that survive the next model release, and the willingness to say no to an LLM when a rules engine is correct.

What a typical AI PM loop actually tests

ROUND	WHAT IT'S REALLY PROBING	WHERE YOU ALREADY HAVE AN ANSWER
Recruiter / HM screen	Can you talk about a real AI product without buzzwords?	The 90-second Prodigy spine (Phase 5)
AI-craft deep dive (evals & failure modes)	Do you actually design evaluation, or just hope demos work?	Phase 4's eval suite sketch, grounded in real SPI-24209 validation design
Product sense case	Given an ambiguous AI feature request, can you scope it, including what NOT to build?	"When NOT to use an agent" (Phase 1), "when classical ML wins" (Phase 0)
Cross-functional / execution	Can you work with eng on architecture trade-offs without hand-waving?	The Product Center AI Architecture case study (Phase 2)

Your unfair advantage, stated plainly

PLAIN ENGLISH

Most AI PM candidates have read about agents. You've shipped agent PRDs (SPI-24205 through 24671), run real discovery interviews with actual issuers (HDFC, Sparrow, Ryze) about an AI product, synthesized cross-issuer findings into a decision-ready matrix, and can walk a reviewer through an approved architecture decision line by line. That combination - real user research plus real agentic system design, in a regulated domain - is exactly the scarce profile hiring managers describe wanting and rarely find.

ANALOGY

There are many people who can describe a car engine from a textbook. Far fewer have actually diagnosed a real engine, with a real owner, under real time pressure, and can show the repair log.

CONCRETE EXAMPLE

"Tell me about a time you scoped an AI feature and decided part of it should NOT be agentic" has a real, specific answer for you: the validation framework is deliberately deterministic, not LLM-judged, precisely to keep release decisions auditable.

In Prodigy

Every phase in this playbook maps to a spec you've already written or a conversation you've already had - this is not hypothetical practice material.

Why this matters as an AI PM

Interviewers filter hard on "have they actually done this" vs. "have they read about this." Lead every answer with the real artefact, not the concept.

Portfolio plan - built from what you've already shipped this week

Decision artefact

1. The Prodigy architecture teardown

Write up the Product Center AI Architecture decision as a case study: the problem, the options considered (embedded vs. external harness, MCP vs. bespoke), the decision, and how you'd evaluate it post-launch. This is your "read code, reason about trade-offs" proof - the PM-grade artefact.

Decision artefact

2. The three-issuer synthesis, reframed as a PRD input

Your HDFC / Sparrow / Ryze synthesis is already a real discovery-to-roadmap artefact. Add one section: "if I had to ship one Prodigy concept next quarter based on this evidence, here's the eval I'd run before calling it done." That one section turns user research into AI-PM-specific proof.

Built artefact

3. A small, real eval harness

Pick one Prodigy skill (e.g. guided configuration). Write 5 golden conversations and 5 release-blocking negative tests by hand, in a spreadsheet or a short script. You don't need production infra - you need to show you can specify what "correct" means before a model exists to test.

Positioning

4. One CFO-language explainer per hard concept

For 3-4 concepts in this playbook (grounding, guardrails, eval design, propose-vs-apply), write one page explaining it to a hypothetical non-technical exec. This is the best retention technique available, doubles as interview rehearsal, and is a ready-made LinkedIn/writing sample.

A NOTE ON SEQUENCING (FROM THE ROADMAP THIS REWORKS)

The Ground-Up Roadmap explicitly says proof-of-work should start in week 8, not week 22 - most candidates run out of time because they treat the portfolio as the last step instead of the parallel track. You're already past week 8's version of "have I shipped anything real" - the honest next move is packaging what exists, not generating new material from scratch.

Interview-safe framing for the pivot itself

IF ASKED "WHY AI PM, AND WHY NOW?"

"I've been the PM shipping the agent layer on top of a real system of record - not reading about agents, designing the skill boundaries, the confirm-before-write gates, and the evals for one. I've also run the discovery interviews that fed that roadmap. What I want next is to do that at a place where the AI product *is* the product, and to go deeper on the eval-design and model-trade-off muscles that a platform team like mine only exercises at the edges."

Self-check before you apply

☐ I can name one feature I deliberately kept non-agentic, and why.

☐ I can sketch a golden conversation and a release-blocking negative test from memory.

☐ I can explain propose-vs-apply without using the word "AI" more than twice.

☐ I have one written case study a stranger could read in 5 minutes and understand my judgment, not just my vocabulary.

☐ I can say "I don't know, here's how I'd find out" for at least one adjacent topic (GNNs, fine-tuning, multimodal) without bluffing.

Speak Prodigy in agent vocabulary

Goal: 60-90 second answers. No fluff. No overclaim.

Ready spoken answers

1 - WHAT IS AN AGENT IN YOUR PRODUCT?

"In Prodigy an agent is a goal-seeking loop on the OpenAI Agents SDK that can call typed tools, observe results, and continue under stop conditions. It is not a free-form chatbot with production keys. For configure, the goal is a review-ready draft; tools load schema and lookups; writes wait for Alex to confirm; apply waits for change request and human deploy."

2 - SKILL VS. TOOL VS. MCP?

"A skill is a versioned procedure with confirmation points - how to run blueprint or guided configure. A tool is a typed function in ToolRegistry that hits the same application services as REST. MCP is the protocol so an external harness can discover and call those tools over Streamable HTTP with OAuth - not a second source of truth and not approve/apply."

3 - WHY NOT LET THE AGENT DEPLOY?

"Because blast radius in issuer product config is institutional. We separate propose, form truth, CR validation, dry run, and human deploy. Agent adapters intentionally expose read, validate, and propose only. That's how we stay bank-grade without pretending the model is a release manager."

4 - HOW DO YOU STOP HALLUCINATIONS?

"Schema is law. Lookups come from list APIs. Empty or error means we say so. Citations for Q&A come from GitDB or service results. Document extractors show a confirm board. Evals include release-blocking negatives for invented options, cross-tenant access, and writes before confirm."

5 - CHATBOT VS. AGENT - HOW DID YOU DECIDE?

"Cited fee questions are chat plus read tools. Validation is a deterministic workflow. Launch-from-MITC is agentic because language and multi-step tools matter - but still human-in-the-loop. I use a simple test: is the path fixed, is the write dangerous, is language the interface?"

6 - MULTI-AGENT - DID YOU NEED IT?

"We bias to a single embedded agent with routed skills: create 24205, update 24213, ask 24214, clone 24207. Multi-agent is on the table for isolation or parallel specialists, but each added loop costs latency and failure modes. I won't add agents for resume keywords."

7 - TELL ME ABOUT EVALS.

"Each skill gets golden conversations - expected tools and outcomes. Release blockers include cross-tenant, unauthorized tools, prompt injection via uploads, stale revisions, and pre-confirm mutation. Validation engine results are not 'LLM judged' into green."

8 - BUILD VS. BUY FOR THE HARNESS?

"We buy model access and an Agents SDK/runtime via Azure Foundry rather than training a foundation model. We build the product contracts: skills, ToolRegistry, entitlements parity, CR integration, and UX. Differentiation is the control plane for issuer products, not the base model."

9 - HOW DOES PAGE-AWARE CONTEXT WORK?

"The base passes page, product, blueprint, active tab, and session into skills so Alex doesn't restate the screen. That's context engineering as a product feature - it reduces tokens and errors versus megaprompts."

10 - SOFT FAILURE MODE?

"If Prodigy is unavailable, Product Center forms and existing CR/deploy workflows still operate. AI is an acceleration layer on the system of record, not a fork of it."

Frameworks to keep in your pocket

Build vs. buy

Buy: models, Agents SDK, Foundry hosting. Build: skills, tools, entitlements, CR glue, evals, UX. Buy commodity intelligence; build the control plane.

Agent vs. workflow

Agent when language -> multi-step tools under uncertainty. Workflow when pass/fail must be identical and auditable. Hybrid is normal.

Trap questions - answer without overclaiming

TRAP	WEAK ANSWER	STRONG ANSWER
"So the AI learns from every bank?"	"Yes, it trains on chats."	"Inference uses tenant-scoped tools and data; learning needs governed pipelines. We don't casually train on customer chat."
"Can it just configure everything?"	"Fully autonomous."	"It proposes; humans confirm fields; CR/deploy apply. Autonomy stops where blast radius starts."
"Is MCP live for all journeys?"	Invent status.	"Architecture exposes MCP + AI REST sharing ToolRegistry; embedded UI is the primary shipped harness - I state live scope honestly."
"How do you guarantee zero hallucination?"	"Temperature 0."	"We don't claim zero fluency errors; we claim schema/tools/confirm/evals make harmful config hallucination a release-blocking defect."
"Why not multi-agent swarm?"	"We have 15 agents."	"Routed skills first; multi-agent only for isolation/parallelism with a cost case."

Mock interviewer drill

Drill A. "Draw Prodigy's apply path and mark where the LLM has authority."

Expect: authority only to propose/explain; stars on confirm and human deploy.

Drill B. "A VP asks for fully autonomous product launch by Friday." Respond in 60 seconds with principles, not panic.

Drill C. "What's the difference between your validation framework and the NLP summary?"

Drill D. "Show me a negative eval that would block my release."

ONE-PAGE NARRATIVE - SHIPPING AGENTIC PRODUCT SAFELY IN A BANK

"Start from the system of record, not the model. Define journeys as skills with explicit confirmation points. Expose only read/validate/propose tools that call the same services as REST. Match entitlements to the UI and fail closed. Keep apply inside existing CR, review, dry run, and human deploy. Ground answers in GitDB/service citations. Treat uploads as injection surfaces. Measure with golden conversations and release-blocking negatives. Prefer one routed agent over a swarm. If the assistant dies, the forms still work. That's Prodigy: intelligence as a layer, governance as the product."

Common misconceptions (kill these early)

MISCONCEPTION	REALITY IN PRODIGY
"AI features replace Product Center forms."	Chat sits beside forms; forms remain truth.
"Skills are permissions."	Skills are procedures; entitlements fail closed separately.
"MCP means the model is fine-tuned on our data."	MCP is a tool-access protocol for harnesses.
"Temperature 0 eliminates all risk."	Risk is managed by tools, schema, confirm, evals, CR.
"More agents = more intelligence."	More agents = more coordination tax unless isolation requires them.

MISCONCEPTION	REALITY IN PRODIGY
"NLP summary is the validation."	NLP explains; the validation engine decides block/warn.
"Citations can be 'as the model recalls'."	Citations need GitDB/service evidence.
"Dry run is optional theater."	Dry run is a deliberate stop before live mutation.
"If the demo worked, evals are optional."	Demos are anecdotes; negatives block releases.
"An AI PM needs to be able to train models."	Neither AI PM nor AI FDE roles expect that - they expect correct intuitions about failure modes and the ability to translate business <-> system design.

Regalia Gold US - canonical examples to reuse

- Launch US retail credit -> blueprint US Retail Credit -> name Regalia Gold US.
- Joining fee \$95 proposed from guided ask or PDF; form updates only after confirm.
- Annual fee \$150 as a policy lookup - match existing or create-new; don't paint a raw number onto the wrong binding.
- Quick update \$95 -> \$150 from home without a blank create.
- Q&A: current fee + history with citations; optional handoff to update.
- CR: section NLP before->after; strict validation blocks; impact with assumptions labeled.
- Deploy: last deployed vs. pending master; dry run; human deploy.

Glossary - interview-safe one-liners

TERM	ONE-LINER
LLM	Large model that predicts tokens; useful for language and planning, not a database.
Token	Subword unit of text the model reads/writes; bills and context use tokens.

TERM	ONE-LINER
Embedding	A vector representation of text where similar meaning = nearby points.
Transformer / attention	The architecture that decides, per token, how much to weigh every other token in context.
Context window	Max tokens per call for instructions, history, and tool results.
Inference	Running a model to produce an output; chatting is inference.
Training	Updating weights from data; not what happens when Alex chats.
Fine-tuning	Adjusting weights on your own examples; changes behaviour, not facts - last resort after prompting + RAG.
Sampling / temperature	Controls randomness in decoding; variance is why we confirm writes.
Prompting	Instructions for a call.
Context engineering	System design of everything the model sees: skills, tools, state, retrieval.
Chatbot	Conversational UI without a strong tool-operating loop.
Agent	Goal-seeking loop that acts with tools, observes, repeats, then stops.

TERM	ONE-LINER
Tool / function calling	Typed function the harness executes on the model's request.
ToolRegistry	Catalog of typed tools; handlers share services with REST.
Skill	Versioned procedure (name, description, steps, confirm points).
RAG	Retrieve relevant data into context before generating; grounds answers in your facts.
Vector database	Storage/search engine for embeddings, at scale - not always needed.
MCP	Model Context Protocol - standard way for external agents to use tools.
Streamable HTTP	MCP transport style for server communication.
OAuth resource	Protected API identity model for authorized MCP/REST access.
Embedded agent	In-product harness via AI REST + UI context.
External harness	Outside client speaking MCP (or similar) to the same tools.
Handoff	Transfer of goal + context to another skill/agent.
Router	Intent classifier that picks the owning skill.
Orchestrator-worker	Planner delegates subgoals to specialists and merges.

TERM	ONE-LINER
HITL	Human-in-the-loop approval on material actions.
Confirm-before-write	Propose pending value; patch only after confirm.
Propose ≠ apply	Form proposals ≠ production apply; CR/deploy applies.
Schema-as-law	Blueprint fields/enums/conditionals define valid config.
Hallucination	Fluent false claim; dangerous when it mutates config.
Citation	Pointer to GitDB/service evidence for a claim.
Guardrail	Input/output/action constraint around the model.
Prompt injection	Hostile text trying to override instructions via user/document content.
Eval	Systematic test of agent behaviour; goldens + negatives.
Golden conversation	Scripted happy path with expected tools/outcomes.
LLM-as-judge	Using a model to score open-ended output quality, anchored to human-reviewed references.
Fail closed	On auth/tool failure, deny rather than invent or leak.
Entitlement parity	Agent can only do what the user's UI already allows.

TERM	ONE-LINER
Unit economics (AI)	Cost and latency per successful journey - a product KPI, not just an infra metric.
GNN	Graph neural network - for when relationships between records matter more than the records.
Multimodal	One model handling more than one input type (text, image, audio).
Azure Foundry	Enterprise hosting path for model/API usage in this architecture.
Agents SDK	Runtime helping implement the tool-calling agent loop.
Dry run	Pre-deploy simulation without live mutation.
System of record	Authoritative store/workflow - here Product Center / GitDB / CR.
AI Product Manager	Owns a product whose behaviour is probabilistic; graded on eval design, trade-off judgement, and roadmap calls that survive the next model release.
AI Forward Deployed Engineer	Embedded with a customer, ships production code that wires AI into their real, messy systems.

Can you defend Prodigy - and yourself as an AI PM?

Tick only what you can explain out loud in under two minutes with a Regalia Gold US example.

☐ Nest AI -> ML -> DL -> GenAI -> LLM without hand-waving

☐ Explain embeddings and attention in one sentence each, without maths

☐ Explain tokens, context packing, and why page-aware context is a product feature

☐ Separate training vs. inference; refuse "chat teaches the model" overclaim

☐ Decide chatbot vs. workflow vs. agent for a new ask

☐ Draw the agent loop and name stop conditions

☐ Explain confirm-before-write and propose $\neq$ apply

☐ Define skill, tool, ToolRegistry, MCP, embedded vs. external

☐ Explain RAG, when you don't need a vector DB, and the fine-tuning decision framework

☐ Recite the architecture: AI REST, MCP, Agents SDK, Azure Foundry, no approve/apply adapters

☐ Route create/update/ask/clone to 24205/24213/24214/24207

☐ Argue single agent + skills vs. multi-agent with cost/blast-radius reasoning

☐ Sketch a golden conversation and a release-blocking negative test unaided

☐ Explain unit economics: cost and latency per successful journey

☐ Walk Path 1 -> CR -> dry run -> human deploy in agent vocabulary

☐ Deliver the 90-second spine story without notes

☐ Answer trap questions without inventing live scope

☐ Name your two Phase 6 portfolio artefacts and where they'd live in a portfolio site

PASS BAR

If you can teach Phase 2, Phase 4, and Phase 6 to another PM on a whiteboard, you're ready for an AI Product Manager loop - not just ready to talk about Prodigy.